

DOCUMENTAÇÃO DO PROJETO

Sistema de Gerenciamento de Eventos

Projeto desenvolvido em Java utilizando Programação Orientada a Objetos (POO)

Curso: Sistemas de Informação

Instituição: IFBA — Campus Vitória da Conquista

Integrantes:

Ryan Lucas Silva Dias

William de Jesus Fernandes

1. Problema

A organização de eventos envolve o controle de participantes, inscrições, palestrantes e atividades. Quando essas informações são controladas manualmente, podem ocorrer problemas como inscrições duplicadas, excesso de participantes e conflitos entre atividades.

Dessa forma, foi desenvolvido um sistema para auxiliar no gerenciamento dessas informações e aplicar regras que garantam o funcionamento adequado do evento.

2. Objetivo

O objetivo do projeto é desenvolver um sistema orientado a objetos capaz de gerenciar eventos, participantes, inscrições, palestrantes e atividades.

O sistema foi desenvolvido utilizando conceitos de Programação Orientada a Objetos, como classes, objetos, encapsulamento, construtores, métodos e relacionamentos entre objetos.

3. Funcionalidades

- Cadastro de participantes.
- Cadastro de palestrantes.
- Criação de eventos.
- Criação de atividades.
- Associação de atividades a eventos.
- Associação de palestrantes às atividades.
- Inscrição de participantes em eventos.
- Controle da capacidade máxima do evento.
- Impedimento de inscrições duplicadas.
- Impedimento de atividades no mesmo horário e local.
- Busca de participante utilizando recursão.

4. Classes do Sistema

4.1 Participante

A classe Participante representa uma pessoa que pode participar de um evento. Possui os atributos nome e CPF, além de um construtor e métodos get para acessar as informações.

4.2 Palestrante

A classe Palestrante representa uma pessoa responsável por participar de uma ou mais atividades. Possui os atributos nome e especialidade, além de construtor e métodos de acesso.

4.3 Inscricao

A classe Inscricao representa a inscrição de um participante em um determinado evento. Possui referências para um Participante e um Evento.

4.4 Atividade

A classe Atividade representa uma atividade realizada durante um evento. Possui nome, horário, duração, local, palestrante e evento.

4.5 Evento

A classe Evento é responsável pelo gerenciamento das inscrições e atividades. Possui nome, local, capacidade máxima, lista de inscrições e lista de atividades. Também contém métodos para adicionar participantes, adicionar atividades e realizar a busca recursiva.

5. Relacionamentos entre as Classes

- Um Participante pode possuir várias Inscrições.
- Uma Inscrição está relacionada a um Participante e a um Evento.
- Um Evento pode possuir várias Inscrições.
- Um Evento pode possuir várias Atividades.
- Uma Atividade está associada a um Evento.
- Uma Atividade possui um Palestrante.
- Um Palestrante pode participar de várias Atividades.

6. Regras de Negócio

Regra 1 — Inscrição duplicada: um participante não pode se inscrever duas vezes no mesmo evento. O sistema verifica o CPF antes de criar uma nova inscrição.

Regra 2 — Capacidade máxima: o evento possui uma capacidade máxima de participantes. Quando essa capacidade é atingida, novas inscrições são bloqueadas.

Regra 3 — Associação da atividade: toda atividade criada no sistema está associada a um evento.

Regra 4 — Palestrante: um mesmo palestrante pode participar de várias atividades.

Regra 5 — Horário e duração: toda atividade possui um horário e uma duração definidos.

Regra 6 — Conflito de horário e local: duas atividades diferentes não podem ocupar o mesmo horário e local dentro do evento.

7. Recursão

O sistema utiliza recursão na busca de participantes. O método buscarParticipante chama um método auxiliar chamado buscarParticipanteRecursivo, que percorre a lista de inscrições uma posição por vez.

Quando o participante é encontrado, o método retorna o objeto correspondente. Caso o índice alcance o final da lista, o método retorna null. Dessa forma, a recursão é utilizada de maneira coerente para realizar a busca.

8. Encapsulamento

O encapsulamento é utilizado por meio da declaração dos atributos como private. Dessa forma, os atributos das classes não podem ser acessados diretamente de fora delas.

Para consultar essas informações, são utilizados métodos públicos, como getNome(), getCpf(), getLocal() e outros métodos de acesso. Isso protege os dados internos dos objetos e demonstra o conceito de encapsulamento da Programação Orientada a Objetos.

9. Construtores

As classes possuem construtores responsáveis por inicializar os objetos no momento em que eles são criados.

Por exemplo, ao criar um objeto Evento, são informados o nome, o local e a capacidade máxima. Da mesma forma, participantes, palestrantes, inscrições e atividades recebem seus dados necessários por meio de seus respectivos construtores.

10. Testes Realizados

Durante a execução do sistema foram realizados testes para verificar as principais regras de negócio.

Teste 1: inscrição de um participante — resultado: inscrição realizada com sucesso.

Teste 2: tentativa de inscrição do mesmo participante novamente — resultado: inscrição recusada.

Teste 3: inscrição até atingir a capacidade máxima — resultado: participantes aceitos até o limite.

Teste 4: tentativa de ultrapassar a capacidade — resultado: inscrição recusada e mensagem de evento lotado.

Teste 5: inclusão de uma atividade — resultado: atividade adicionada com sucesso.

Teste 6: inclusão de outra atividade no mesmo horário e local — resultado: atividade recusada.

Teste 7: busca recursiva de participante pelo CPF — resultado: participante encontrado.

11. Dificuldades Encontradas

Durante o desenvolvimento, algumas dificuldades estiveram relacionadas à compreensão dos conceitos de Programação Orientada a Objetos, principalmente no relacionamento entre as classes e no uso de listas de objetos.

Também foi necessário compreender como aplicar as regras de negócio dentro dos métodos da classe Evento, além de utilizar uma busca recursiva de forma coerente.

12. UML

O diagrama UML representa visualmente a estrutura do sistema, mostrando suas classes, atributos, métodos e os relacionamentos existentes entre os objetos.

O modelo UML do projeto deve apresentar as classes Participante, Palestrante, Inscricao, Atividade e Evento, além das respectivas associações e cardinalidades.

ESPAÇO RESERVADO PARA INSERIR O DIAGRAMA UML DO PROJETO.

13. Conclusão

O projeto permitiu aplicar conceitos fundamentais da Programação Orientada a Objetos em um sistema prático de gerenciamento de eventos.

Por meio das classes, objetos, encapsulamento, construtores, relacionamentos, regras de negócio e recursão, foi possível desenvolver um sistema capaz de controlar participantes, inscrições, atividades, eventos e palestrantes.

Além da implementação do código, os testes realizados permitiram verificar o funcionamento das principais regras definidas para o sistema.